

PPAD (PPGCC)

Programação OpenMP

Relatório — Opção 1

Integrante: Gian Franco Joel Condori Luna

Entrega: 19/11

1 Introdução

O objetivo da atividade foi aplicar conceitos de **paralelismo de memória compartilhada**, utilizando a biblioteca **OpenMP**, em um código-fonte de interesse real, de modo a avaliar ganhos de desempenho e *speedup* obtidos com o uso de múltiplos núcleos de processamento.

Além do objetivo técnico proposto na atividade, este estudo está diretamente relacionado ao problema de pesquisa desenvolvido no meu doutorado, que busca prever setores com maior probabilidade de ocorrência de casos de raiva na cidade de Arequipa - Perú. Para essa finalidade, são utilizados dados geográficos e socioeconômicos dos bairros urbanos, registros históricos de casos de raiva, a localização dos postos de saúde e a distância dos canais de água em relação aos bairros.

A obtenção da distância mínima entre cada bairro e o posto de saúde mais próximo é uma etapa, pois permite construir um grafo de conhecimento em que apenas os bairros localizados a menos de 2000 metros de um posto de saúde são conectadas por uma aresta. Esse grafo servirá como entrada para um modelo de rede neural em grafos (*Graph Neural Network* - GNN), o qual tem como objetivo prever, com base nos padrões espaciais e nas relações entre as entidades, os possíveis setores onde poderão surgir novos casos de raiva no próximo ano.

2 Descrição do código-fonte

O código implementado realiza o cálculo da **distância mínima entre bairros de casas (polígonos) e centros de saúde**, com base em coordenadas extraídas de arquivos no formato **CSV** e **KML**.

Cada bairro está representado por um conjunto de vértices (latitude e longitude) correspondentes a conjuntos de casas, enquanto os centros de saúde representam a localização de postos de saúde. O programa lê todos os arquivos de entrada, constrói a estrutura de dados correspondente e, para cada polígono, calcula a menor distância até qualquer centro de saúde, utilizando a **fórmula de Haversine**.

Essa rotina é computacionalmente intensiva, pois envolve o cálculo de distâncias geodésicas, portanto o trecho principal do cálculo foi, paralelizado com **OpenMP**, visando reduzir o tempo total de execução.

Estrutura dos arquivos

- **main.cpp** — contém o código principal: leitura de arquivos, cálculo das distâncias e medição de tempo.
- **geodata.cpp** — implementa as funções auxiliares de leitura de dados (CSV/KML) e a função de distância Haversine.
- **geodata.h** — define as estruturas e protótipos das funções.
- **dataset/Economic_income_cluster-selected** — arquivos CSV contendo os pontos que compõem os polígonos dos bairros de casas..
- **dataset/Puestos_de_salud_AQP_12ene2024.kml** — arquivo KML contendo a localização dos postos de saúde em Arequipa.

Principais funções

- `cargarPoligonos()` — lê os arquivos CSV contendo coordenadas de múltiplos polígonos.
- `cargarCentros()` — extrai as coordenadas de postos de saúde de um arquivo KML.
- `haversine()` — calcula a distância entre dois pontos na superfície terrestre.
- `distancia_minima()` — determina a menor distância entre os vértices de um polígono e todos os centros.

O cálculo principal é realizado de forma paralela conforme o trecho abaixo:

```
#pragma omp parallel for schedule(dynamic)
for (int i = 0; i < (int)cuadras.size(); ++i) {
    dist_min[i] = distancia_minima(cuadras[i], centros);
}
```

3 Plataforma de execução

Os experimentos foram realizados em um equipamento pessoal com as seguintes características:

- **Processador:** Intel Core i7-11800H (codinome *Tiger Lake*, 8 núcleos físicos / 16 threads)
- **Memória RAM:** 16 GB DDR4
- **Sistema operacional:** Ubuntu 24.04 LTS (64 bits)
- **Compilador:** g++ 13.2 com suporte a OpenMP (`-fopenmp`)

4 Execução do código

O código pode ser compilado e executado em qualquer ambiente Linux com suporte à biblioteca OpenMP. A seguir, descreve-se o procedimento utilizado:

1. Certifique-se de que o compilador g++ instalado possui suporte a OpenMP. Isso pode ser verificado com o comando:

```
g++ --version
```

2. Compile o código-fonte utilizando a flag `-fopenmp`:

```
g++ -fopenmp main.cpp geodata.cpp -o distancias
```

3. Execute o programa:

```
./distancias
```

4. O programa exibirá mensagens de progresso, o número de polígonos e centros carregados, além do tempo total de execução para cada configuração de número de *threads*.
5. Para alterar o número de *threads* utilizadas, defina a variável de ambiente `OMP_NUM_THREADS` antes da execução. Por exemplo:

```
export OMP_NUM_THREADS=8
./distancias
```

5 Metodologia de medição

O desempenho foi medido por meio da função `omp_get_wtime()`, registrando o tempo total de execução do cálculo principal, com diferentes números de *threads*. Para cada configuração, foram realizados três experimentos, tomando-se a média dos tempos obtidos.

Configurações testadas

Execução	Nº de <i>threads</i>	Descrição
1	1	Execução sequencial (baseline)
2	2	Paralelismo parcial
3	4	Uso de metade dos núcleos
4	8	Uso total dos núcleos físicos
5	16	Uso total de <i>threads</i> lógicas (hyper-threading)

6 Resultados obtidos

Nº de <i>threads</i>	Tempo (s)	<i>Speedup</i>	Eficiência (%)
1	1.32735	1.00	100.0
2	0.626315	2.12	106.0
4	0.406590	3.26	81.5
8	0.381234	3.48	43.5
16	0.380276	3.49	21.8

O *speedup* foi calculado como:

$$\text{Speedup} = \frac{T_{seq}}{T_{par}}$$

e a eficiência como:

$$\text{Eficiência} = \frac{\text{Speedup}}{N_{threads}} \times 100$$

7 Discussão dos resultados

Os resultados obtidos indicam que a paralelização do código com OpenMP trouxe uma melhora significativa no tempo de execução em relação à versão sequencial. O tempo total de execução reduziu-se de aproximadamente 1,33 segundos (para um único *thread*) para cerca de 0,38 segundos com 16 *threads*.

Observa-se um ganho quase linear até aproximadamente 4 *threads*, com um *speedup* de 3,26 vezes e eficiência de 81,5%. No entanto, ao aumentar o número de *threads* para 8 e 16, o ganho marginal torna-se reduzido, indicando a saturação dos recursos de paralelismo disponíveis no processador e o impacto das seções de código não paralelizáveis.

O leve aumento de eficiência acima de 100% para dois *threads* pode ser explicado por variações de cache, escalonamento do sistema operacional e sobreposição de tarefas, efeitos comuns em medições de curta duração.

8 Conclusões

O estudo realizado evidenciou a eficiência do uso de OpenMP para acelerar o cálculo das distâncias mínimas entre polígonos e centros de saúde, problema originalmente sequencial. A análise de desempenho mostrou um ganho expressivo até quatro *threads*, com redução de mais de 65% no tempo de execução em relação à versão sequencial.

A partir de oito *threads*, observou-se uma diminuição na taxa de aceleração, resultado esperado devido à sobrecarga de criação e sincronização das *threads* e à limitação do paralelismo intrínseco do problema.

Ainda assim, o tempo total permaneceu estável, confirmando a boa escalabilidade até o limite de eficiência do hardware utilizado.

Como trabalho futuro, seria interessante avaliar o comportamento da aplicação com conjuntos de dados maiores e testar outras estratégias de paralelização, como o uso de bibliotecas híbridas (OpenMP + MPI), de forma a maximizar o aproveitamento dos núcleos de processamento disponíveis.

Referências

- [1] OpenMP Architecture Review Board. (2023). *OpenMP Application Programming Interface – Version 5.2*. Disponível em: <https://www.openmp.org/specifications/>
- [2] OpenAI. (2025). *ChatGPT (GPT-5) [modelo de linguagem de grande escala]*. Disponível em: <https://chat.openai.com/>